

Implement Repository Pattern (Two Phases)

Scenario

You are building a **Console-based Library Book Management System** for a training institute.

Initially, the institute **does not want to use a database**.
The application should run using **temporary in-memory storage**.

Later, management decides to **store the data permanently in SQL Server**.

The system should be designed in such a way that **switching the storage implementation from In-Memory → SQL Server requires minimal code changes**.

Your task is to implement the system using the **Repository Pattern** so that the **data access layer can be replaced easily without affecting the rest of the application**.

Phase 1 — Repository Without Database

Create an **ASP.NET Core MVC application** that manages books using **in-memory storage**.

1. Model

Create a model class named:

Book

Properties

Property	Type
BookId	int
Title	string
Author	string
Price	decimal

2. Repository Interface

Create an interface named:

```
IBookRepository
```

Methods

```
GetAllBooks()  
GetBookById(int id)  
AddBook(Book book)  
DeleteBook(int id)
```

Important Rule

! Do NOT store **List<Book>** inside the controller

All data operations must go through the **repository interface**.

3. In-Memory Repository Implementation

Create a class named:

```
MemoryBookRepository
```

This class must implement:

```
IBookRepository
```

Requirements

Store books **in memory**.

You may use any of the following:

```
Dictionary<int, Book>  
Array  
List<Book>  
Any suitable data structure
```

Initial Data

Initialize the repository with **3 sample books**.

Example:

```
Clean Code — Robert C. Martin  
Design Patterns — GoF  
Refactoring — Martin Fowler
```

4. Controller

Create a controller named:

```
BookController
```

Required Actions

Action	Purpose
List	Display all books
Details	Display a single book
Create	Add a new book
Delete	Remove a book

Dependency Injection Requirement

The controller must use **constructor injection**.

Example idea:

```
private readonly IBookRepository _repo;  
  
public BookController(IBookRepository repo)  
{  
    _repo = repo;  
}
```

Expected Result (Phase 1)

The application should run **without SQL Server** and support:

- List Books
 - View Book Details
 - Add Book
 - Delete Book
-

Note

All data will be stored **in memory**.

When the application **restarts**, all books will be **lost**.

Phase 2 — Migrate Repository to SQL Server

Management now wants to **store the data permanently in SQL Server**.

You must extend the **same project** to support **Entity Framework Core**.

Important Rule

! Do NOT change controller logic

Only the **repository implementation** should change.

Step 1 — Create DbContext

Create a class:

```
LibraryDbContext
```

Add a DbSet:

```
DbSet<Book> Books
```

Step 2 — SQL Repository Implementation

Create another repository implementation:

`SqlBookRepository`

This class must implement:

`IBookRepository`

Use **Entity Framework Core** to perform:

- Retrieve all books
 - Retrieve a book by ID
 - Add a new book
 - Delete a book
-

Expected Result (Phase 2)

After implementing the SQL repository:

The application should now:

- Store books in **SQL Server**
 - Use **Entity Framework Core**
 - Work with the **same controllers and views**
 - Require **only a repository implementation change**
-

Core Concept Being Tested

This assignment evaluates your understanding of:

- Repository Pattern
- Dependency Injection
- Abstraction
- Entity Framework Core
- Clean Architecture principles

The key idea:

`Controller` → `IBookRepository` → `Implementation`
Implementation can change without affecting controller logic.